LAB#09

SUPERVISED LEARNING ALGORITHMS

Perform supervised learning Decision Tree and Naïve Bayes algorithms for training, testing and classification using Python.

I. Supervised Learning

Supervised learning as the name indicates the presence of a supervisor as a teacher. Basically, supervised learning is a learning in which we teach or train the machine using data which is well labeled that means some data is already tagged with the correct answer. After that, the machine is provided with a new set of examples (data) so that supervised learning algorithm analyses the training data (set of training examples) and produces a correct outcome from labeled data.

For instance, suppose you are given a basket filled with different kinds of fruits. Now the first step is to train the machine with all different fruits one by one like this:

- If shape of object is rounded and depression at top having color Red then it will be labelled as –**Apple**.
- If shape of object is long curving cylinder having color Green-Yellow then it will be labelled as –**Banana**.

Now suppose that we have trained the dataset and were given a separate fruit say banana for instance.

Since the machine has already learned the things from previous data and this time have to use it wisely. It will first classify the fruit with its shape and color and would confirm the fruit name as BANANA and put it in Banana category. Thus, the machine learns the things from training data (basket containing fruits) and then apply the knowledge to test data (new fruit).

Supervised learning classified into two categories of algorithms:

- **Classification**: A classification problem is when the output variable is a category, such as "Red" or "blue" or "disease" and "no disease".
- **Regression**: A regression problem is when the output variable is a real value, such as "dollars" or "weight".

II. Decision Tree Algorithm (DTS)

Decision tree is the most powerful and popular tool for classification and prediction. A Decision tree is a flowchart like tree structure, where each internal node denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (terminal node) holds a class label.

a) Construction of the Decision Tree:

A tree can be *"learned"* by splitting the source set into subsets based on an attribute value test.

This process is repeated on each derived subset in a recursive manner called *recursive partitioning*. The recursion is completed when the subset at a node all has the same value of the target variable, or when splitting no longer adds value to the predictions. The construction of decision tree classifier does not require any domain knowledge or parameter setting, and therefore is appropriate for exploratory knowledge discovery. Decision trees can handle high dimensional data. In general decision tree classifier has good accuracy. Decision tree induction is a typical inductive approach to learn knowledge on classification.

b) Decision Tree Representation:

Decision trees classify instances by sorting them down the tree from the root to some leaf node, which provides the classification of the instance. An instance is classified by starting at the root node of the tree, testing the attribute specified by this node, and then moving down the tree branch corresponding to the value of the attribute. This process is then repeated for the sub-tree rooted at the new node.

The decision tree in above figure classifies a particular morning according to whether it is suitable for playing tennis and returning the classification associated with the particular leaf. (In this case Yes or No).

c) Used Python Packages:

- **Sklearn:** In python, sklearn is a machine learning package which include a lot of ML algorithms.
- **NumPy:** It is a numeric python module which provides fast math functions for calculations. It is used to read data in NumPy arrays and for manipulation purpose.
- **Pandas:** Used to read and write different files. Data manipulation can be done easily with data frames.

d) Pseudo code for Decision Tree:

- Import the appropriate libraries, Pandas, numpy and sklearn using,
  ```
  import pandas as pd
  import numpy as np
  from sklearn.tree import DecisionTreeClassifier
  ```
- Construct panda data frame.
- Generate the list of features.
- Generate the list of labels. (Note: Number of labels must be same as the number of feature rows)
- Call the chosen classifier container that will train the feature data frame.
  ```
  Classifier =  DecisionTreeClassifier()
  ```
- Train the model using features and label,
  ```
  classifier = classifier.fit(features_1, labels)
  ```
- Include a new entry having only features.
- Predict the class of the new entry using,
  ```
  prediction = classifier.predict(new_entry)
  ```

III. Naive Bayes Classifier

Naive Bayes is a statistical classification technique based on Bayes Theorem. It is one of the simplest supervised learning algorithms. Naive Bayes classifier is the fast, accurate and reliable algorithm. Naive Bayes classifiers have high accuracy and speed on large datasets.
Naive Bayes classifier assumes that the effect of a particular feature in a class is independent of other features. For example, a loan applicant is desirable or not depending on his/her income, previous loan and transaction history, age, and location. Even if these features are interdependent, these features are still considered independently. This assumption simplifies computation, and that's why it is considered as naive. This assumption is called class conditional independence.

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

- P(h): the probability of hypothesis h being true (regardless of the data). This is known as the prior probability of h.
- P(D): the probability of the data (regardless of the hypothesis). This is known as the prior probability.
- P(h|D): the probability of hypothesis h given the data D. This is known as posterior probability.
- P(D|h): the probability of data D given that the hypothesis h was true. This is known as posterior probability.

a) Pseudocode for Naïve Bayes Classifier:

- Import the required libraries.

```
from sklearn import preprocessing
from sklearn.naive_bayes import GaussianNB
```

- Assign features and label variables.
- Perform label encoding on all columns. Scikit-learn provides LabelEncoder library for encoding. It is used to convert categorical data, or text data, into numbers, which our predictive models can better understand.

  ```
  #creating labelEncoder
  le = preprocessing.LabelEncoder()
  # Converting string into numbers.
  weather_encoded=le.fit_transform(weather)
  ```

- Combine all the features in a single variable (list of tuples).
  features=list(zip(feature1_encoded,feature2_encoded,….))
- Generate a model using naive bayes classifier in the following steps:
  1. Create naïve bayes classifier.
     ```
     model = GaussianNB()
     ```

2. Fit the dataset on classifier.
   model.fit(features,label)
3. Perform prediction.
   predicted= model.predict(test_data)

Activity- 1:

Implement the Decision tree algorithm on the data given in the table 9.1 and predict whether the players can play or not when the weather is overcast and the temperature is mild. Also verify results by solving manually.

Table 9.1 Weather Forecast

Weather	Temperature	Play
Sunny	Hot	Yes
Sunny	Hot	Yes
Overcast	Hot	No
Rain	Mild	Yes
Sunny	Cool	No
Overcast	Cool	No
Rain	Mild	Yes
Rain	Mild	Yes
Overcast	Hot	No
Sunny	Hot	No

Activity- 2:

Implement the Decision tree algorithm on any dataset taken from Kaggle.com and predict the new entry entered by the user.

Activity- 3:

Implement Naïve Bayes Algorithm on the dataset given in Table 9.2, to predict whether the players can play or not when the Outlook is Rain, Temperature is Cool, Humidity is High and the Wind is Strong. Also verify results by solving manually.

Table 9.2 Golf play chart

Day	Outlook	Temperature	Humidity	Wind	Play golf
D1	Sunny	Hot	High	Weak	Yes
D2	Sunny	Hot	High	Strong	Yes
D3	Overcast	Hot	High	Weak	No
D4	Rain	Mild	High	Weak	Yes
D5	Sunny	Cool	Normal	Weak	No
D6	Overcast	Cool	Normal	Strong	Yes
D7	Rain	Mild	Normal	Strong	No

D8	Rain	Mild	High	Weak	Yes
D9	Overcast	Hot	Normal	Weak	No
D10	Sunny	Hot	Normal	Strong	No

Activity- 4:

Consider the following dataset given in Table 9.3. Implement Naïve Bayes Algorithm to classify youth/medium/yes/fair**.** Also verify results by solving manually.

Table 9.3 Student income chart

age	*income*	*student*	*credit_rating*	*Class: buys_computer*
youth	high	no	fair	no
youth	high	no	excellent	no
middle_aged	high	no	fair	yes
senior	medium	no	fair	yes
senior	low	yes	fair	yes
senior	low	yes	excellent	no
middle_aged	low	yes	excellent	yes
youth	medium	no	fair	no
youth	low	yes	fair	yes
senior	medium	yes	fair	yes
youth	medium	yes	excellent	yes
middle_aged	medium	no	excellent	yes
middle_aged	high	yes	fair	yes
senior	medium	no	excellent	no

Activity- 5:

Implement the Naïve Bayes algorithm on any dataset taken from Kaggle.com and predict the new entry entered by the user.